

IFN636 Software Life Cycle Management

Assessment 1

XRLend

XR Headset Borrowing System

Full name	Mohamadali Rezaeimanesh
Student ID	n12691631
Tutor's name	Dr Ranesh Naha
Tutorial day and time	Thursdays 11am – 1pm
Project name	XRLend — XR Headset Borrowing System

JIRA	https://connect-team-dvb29u8v.atlassian.net/jira/software/projects/XR/boards/2/backlog
Draw.io	https://drive.google.com/drive/folders/1CBP1RrMoTginZWT9nPN4r3DK1Zh8bRCK?usp=sharing
Figma prototype	https://www.figma.com/proto/LRvpQHryRuvmx7NYTdoBcf/Untitled?node-id=3-1514&starting-point-node-id=3%3A1514
Figma design	https://www.figma.com/design/LRvpQHryRuvmx7NYTdoBcf/Untitled?node-id=0-1
GitHub	https://github.com/ALIRZM/XR-Lend
EC2 instance	i-0e07c3de4b0f61dad (Ali_rzm)

Table of Contents

XRLend — XR Headset Borrowing System	2
IFN636 Software Life Cycle Management	2
TASK 1.....	2
1.1 The Problem	2
1.2 Stakeholders	2
1.3 User Roles	3
1.4 The Two End-to-End Workflows	3
1.5 Scope.....	4
1.6 Success Criteria.....	5
1.7 Requirements List.....	6
1.8 Product Backlog.....	7
1.9 Sprint Plan	11
1.10 Decision and Change Log	16
TASK 2 — SYSTEM DESIGN	17
2.1 The Diagrams.....	17
2.2 Traceability Matrix	20
TASK 3 — UI/UX DESIGN	22
3.1 Low-fidelity wireframes	22
3.2 High-fidelity prototype	22
3.3 The four states	23
TASK 4 — GIT VERSION CONTROL.....	23
4.1 Branches and pull requests	23
4.2 Commit history	23
4.3 README	24
TASK 5 — SAMPLE APPLICATION AND EC2 DEPLOYMENT	24
5.1 What was built	24
5.2 Deployment.....	25
5.3 Security	25
TASK 6 — GenAI disclosure and report.....	25
6.1 GenAI Use Declaration and Evidence Log.....	25
6.2 Individual Reflection.....	27

XRLend — XR Headset Borrowing System

IFN636 Software Life Cycle Management

Task 1: Problem, Requirements and Project Management

Task 2: System Design

TASK 1

1.1 The Problem

University XR labs hold a small number of expensive headsets that many students need to share. Most labs still run this on a paper sign-out sheet, a whiteboard, or a shared spreadsheet.

This causes four problems:

1. **Double bookings.** Two students book the same headset for overlapping days because nobody can see the full calendar at once.
2. **No record of who has what.** When a headset goes missing, staff cannot say who had it last or when it was due back.
3. **Students waste trips.** A student walks to the lab and sees every headset is already out because availability is not visible before they come to the lab in person.
4. **Damage is untracked.** A headset comes back with a cracked lens and there is no record linking that damage to a specific loan.

The lab technician ends up doing the scheduling by hand and holding the whole picture in their head.

XRLend fixes this. It is one website where the technician keeps the headset list up to date, students see what is actually free for the dates they want, and every loan is recorded from request through to return.

Why this is worth building. The main problem is a scheduling rule, it's not a data entry form. Two loans of the same headset must not overlap in time. That single rule is what a spreadsheet cannot do and what the system can support.

1.2 Stakeholders

Stakeholder	Type	What they want
Students	Main user	See what is free, book it, and know the booking is confirmed
Lab technician	Main user	Stop double bookings, know where every headset is, less manual scheduling

Stakeholder	Type	What they want
Lab manager	Secondary	Usage figures to justify buying more headsets
Teaching staff	Secondary	Confidence that students in their unit can get equipment when needed
University IT	Outside	The system follows normal account and security rules
AWS	Supplier	Service stays available

1.3 User Roles

Two roles. The rules are checked on the server, not just hidden in the interface.

Can do	Student	Technician
Sign up and log in	Yes	Yes
Browse the headset list	Yes	Yes
Check availability for a date range	Yes	Yes
Request a loan	Yes	No
Cancel own pending request	Yes	No
See own loan history	Yes	No
Add, edit or retire a headset	No	Yes
Set a headset to under maintenance	No	Yes
Approve or reject a request	No	Yes
Record a collection	No	Yes
Record a return and its condition	No	Yes
See all loans across all students	No	Yes
Sign up as a technician using the staff code	No	Yes

Students can request a headset. Only the technician can approve it and hand it over. This matches how equipment loans normally work, and fewer rules means fewer things to test.

1.4 The Two End-to-End Workflows

Each workflow belongs to a different role and finishes on its own. They meet at the loan request, which is what makes this one system rather than two.

Workflow A — Borrow a headset (Student)

Step	What happens	Requirement
A1	Student logs in	R5
A2	Browses headsets and enters the dates they want	R10
A3	System shows which headsets are free for those dates	R10,R13

Step	What happens	Requirement
A4	Student requests a loan for one of them	R11
A5	Request sits as Pending until the technician decides	R11
A6	Student sees the decision on their loan list, with the reason if it was rejected	R11

Finished when: the request has been approved or rejected and the student can see which.

Workflow B — Manage the lab (Technician)

Step	What happens	Requirement
B1	Technician logs in	R5
B2	Adds a headset with its model and asset tag	R9
B3	Headset is saved as Available and students can now find it	R9
B4	Technician opens the pending request queue	R12
B5	Checks the dates and approves or rejects, giving a reason if rejected	R12,R13
B6	Queue updates and the student sees the decision	R12

Finished when: the request has a decision recorded against it and the queue no longer shows it.

1.5 Scope

In scope

- Sign up, log in, log out
- Two roles, chosen at sign up, with a staff code required for the technician role.
- Headset records with model, asset tag, status and notes
- Availability search across a date range
- Loan requests with start and end dates
- Approve and reject steps, with a reason on rejection
- Recording collection and return, built after the two workflows
- Overlap checking so one headset cannot be double booked
- Student loan history
- Technician view of all loans
- Input checks on all forms
- Manual deployment to AWS EC2 with written steps

Out of scope

Left out	Why
Payments, fines or deposits	Brings in payment security rules, too big for this unit
Email or SMS alerts	Needs an external mail service and adds a dependency we do not need to show the workflow
University single sign-on	Normal login is enough and keeps the system self-contained
Barcode or QR scanning	Needs hardware we do not have. The asset tag is typed in
Mobile apps	A responsive website is enough

Left out	Why
Booking rooms or desks as well	Headsets only. Adding more equipment types would widen the model without adding new reasoning
Waiting lists	If nothing is free the student picks other dates. A queue adds a lot of rules for little value here
Usage reports and charts	Counting loans is enough. Charts do not test any new rule
Automatic CI/CD	The brief says this is out of scope

Assumptions

1. One AWS region (ap-southeast-2) and one EC2 instance, same as Tutorial 4.
2. Data is stored in MongoDB Atlas, same as Tutorial 4.
3. The lab holds 30 headsets, so availability can be checked without heavy optimisation.
4. Loans are whole days. There are no hourly bookings.
5. Collection and return happen in person, so the system records them but does not control them.
6. One technician is on duty at a time, so two staff will not approve the same request at once.
7. The team is 1 person working about 15 hours a week.
8. The site only needs to be running during the marking period.
9. AWS free tier is enough for a demo with a few users.
10. The lab holds a single staff access code, given out in person by the lab manager. There is no per-person staff invite.

1.6 Success Criteria

Each one can be checked. Vague goals like “easy to use” are not included because you cannot measure them.

ID	Criteria	How to check	Target
SC1	No headset has two approved loans that overlap in time	Query the database for overlapping approved loans	0
SC2	A student can go from login to a submitted request	Time a walkthrough	Under 3 min
SC3	Availability results match the loan records	Book 5 test loans, compare search results against the database	100% match
SC4	Tests pass on the release tag	Run the test suite	All pass
SC5	API is fast enough	Time 20 availability searches, check the 95th percentile	Under 400 ms
SC6	Controller test coverage	Coverage report	70% or more
SC7	Roles are enforced on the server	Try 6 requests with the wrong role token	0 get through
SC8	No passwords or keys in the repo	Search the git history	0 found
SC9	Site is reachable during marking	Load the public URL	200 OK
SC10	Every requirement traced to a commit	Check the traceability matrix 2.2	All of R1–R16

1.7 Requirements List

Sixteen requirements. Twelve say what the system does. Four say how it has to behave. R1 to R8 are the accounts tree. Each step in it is built and tested on its own, so each one is its own requirement.

These IDs are used in the backlog, the diagrams and the traceability matrix. They do not change again.

Functional

ID	Name	Requirement
R1	Sign up	A visitor can create an account and pick a role
R2	Staff code check	Check the staff code before a technician account is made
R3	Roles	The system has two roles, Student and Technician. Every account has exactly one
R4	User authentication	Secure authentication with a hashed password and a session token
R5	Log in	Start a session and open the dashboard for the role
R6	Log out	End the session and clear the token
R7	Password check	Compare the password against the stored hash
R8	Session end	Clear the token so protected pages return to login
R9	Manage headsets	A technician can add, edit, set the status of, and retire a headset
R10	Browse and search	A student can browse headsets and search availability for a date range
R11	Request and track loans	A student can request a loan, see their own loans, and cancel a pending request
R12	Decide and record	A technician can see pending requests, approve or reject with a reason, and record collection and return

Non-functional

ID	Name	Requirement	Backed by
R13	No overlapping loans	Two approved loans for the same headset must never overlap in time	SC1
R14	Security	Roles are checked on the server and passwords are stored as hashes	SC7
R15	Performance	The availability search answers in under 400 ms with 30 headsets	SC5
R16	Usability	A student can go from login to a submitted request in under 3 minutes	SC2

R13 is the rule the system exists for. R10 and R12 come from it, because searching and approving are the only two places it is enforced. R7 comes from R14. R14 comes from R3, because there is no point checking a role on the server unless there is more than one role.

1.8 Product Backlog

Four epics, 16 stories, 85 points. 80 points are Must and 5 are Should. Priority uses MoSCoW. Estimates use 1, 2, 3, 5 and 8.

Epic	Name	Pts	Needs first
1	Foundations	30	Nothing
2	Headset Inventory	13	Epic 1
3	Availability and Requests	24	Epic 2
4	Approvals and Returns	18	Epic 3
	Total	85	

Epic 1 — Foundations · 30 points

Accounts, roles, the database, the server and the tests. A user never sees any of it, but nothing else works without it.

Story 1.1 — Sign up · 5 pts · Must · R1, R2, R3, R4, R7

As a visitor, I want to make an account, so that I can use the system.

Acceptance criteria

- Given valid details and the Student role, when I submit, then my account is created with the student role.
- Given the Technician role and the right staff code, when I submit, then my account is created with the technician role.
- Given the Technician role and a wrong or blank staff code, when I submit, then I see an error and no account is made.
- Given a new account, when I look in the database, then the password is stored as a hash.
- Given any account, when I look at it, then it has exactly one role.

Sub-tasks

- Sign up form, role picker, staff code check against an environment variable, password hashing, user model.

Story 1.2 — Log in and log out · 5 pts · Must · R5, R6, R8

As a user, I want to log in and out, so that only I can reach my own pages.

Acceptance criteria

- Given the right details for a student, when I log in, then I reach the student dashboard.

- Given the right details for a technician, when I log in, then I reach the technician dashboard.
- Given wrong details, when I log in, then I see one message that does not say which field was wrong.
- Given I am logged in, when I log out, then my token is cleared and protected pages send me back to login.

Story 1.3 — Role check middleware · 5 pts · Must · R14

As a technician, I want role checks on the server, so that a student cannot reach my pages by changing the URL.

Acceptance criteria

- Given a student token, when it is sent to a technician route, then the server returns 403.
- Given no token, when a protected route is called, then the server returns 401.
- Given a technician token, when it is sent to a technician route, then the request goes through.

Story 1.4 — Set up the database and EC2 · 5 pts · Must · Task

Acceptance criteria

- MongoDB Atlas is connected and the EC2 public IP is on the access list.
- The EC2 instance runs the app and the public URL loads.
- Inbound rules only open the ports the app needs.

Story 1.5 — Write the README and deployment steps · 5 pts · Must · Task

Acceptance criteria

- The README has setup, a short architecture summary, known limits and the deployment URL.
- The deployment steps are good enough to rebuild the server from scratch.
- No password, key or connection string is anywhere in the repo, and ``.env`` is in ``.gitignore``.

Story 1.6 — Unit tests to 70% coverage · 5 pts · Must · Task

Acceptance criteria

- Controller coverage is 70% or more.
- The seven overlap cases from Story 3.2 all pass.
- The wrong-role test from Story 1.3 passes.

Epic 2 — Headset Inventory · 13 points · Needs Epic 1

The technician side of Workflow B. Headsets have to exist before anyone can borrow one.

Story 2.1 — Add a headset · 5 pts · Must · R9

As a technician, I want to add a headset, so that students can see it and book it.

Acceptance criteria

- Given a model and an asset tag, when I save, then the headset shows in the list as Available.

- Given an asset tag that already exists, when I save, then I see an error and nothing is created.
- Given I am a student, when I call the add route, then the server returns 403.

Sub-tasks

- Headset model, POST route, add form, unique asset tag check.

Story 2.2 — Edit a headset and set its status · 5 pts · Must · R9

As a technician, I want to change a headset record, so that the list stays right.

Acceptance criteria

- Given I change the model or the notes, when I save, then the list shows the new values.
- Given I set a headset to Maintenance, when a student searches, then it does not show up.
- Given a headset has approved future loans, when I set it to Maintenance, then I am warned before it saves.

Story 2.3 — Retire a headset · 3 pts · Must · R9

As a technician, I want to retire a broken headset, so that nobody can book it.

Acceptance criteria

- Given I retire a headset, when a student searches, then it never shows up.
- Given a retired headset, when I open an old loan, then the headset still shows on that record.

Epic 3 — Availability and Requests · 24 points · Needs Epic 2

All of Workflow A. This is where the overlap rule is enforced for the first time.

Story 3.1 — Browse the headset list · 3 pts · Must · R10

As a student, I want to see what headsets exist, so that I know what I can ask for.

Acceptance criteria

- Given I am logged in as a student, when I open the list, then I see model, asset tag and status.
- Given there are no headsets, when I open the list, then I see a message, not a blank page.

Story 3.2 — Search availability for a date range · 8 pts · Must · R10, R13, R15

As a student, I want to enter my dates and see only what is free, so that I do not ask for something already booked.

Acceptance criteria

- Given a start and an end date, when I search, then only headsets with no Pending or Approved loan across those dates show up.
- Given an end date before the start date, when I search, then I see a message and nothing is sent.
- Given nothing is free, when I search, then I see a message suggesting other dates.
- Given 30 headsets in the database, when I search, then the answer comes back in under 400 ms.

Sub-tasks

- Overlap query, availability route, date picker, seven boundary tests.

The seven overlap tests

An existing loan runs day 10 to day 14. A new request must be refused for 8 to 12, 12 to 14, 14 to 18, 8 to 18 and 10 to 15. It must be allowed for 5 to 9 and for 16 to 20.

Story 3.3 — Request a loan · 8 pts · Must · R11, R13, R16

As a student, I want to request a headset for my dates, so that a technician can decide.

Acceptance criteria

- Given a free headset and valid dates, when I submit, then a loan is created with the status Pending.
- Given someone booked the same dates while I was filling in the form, when I submit, then my request is refused with a message saying it is no longer free.
- Given I am a technician, when I call the request route, then the server returns 403.
- Given a new student account, when the path from login to a submitted request is timed, then it takes under 3 minutes.

Sub-tasks

- Loan model, POST route, second overlap check before save, request form.

Story 3.4 — See my loans and cancel a pending request · 5 pts · Must · R11

As a student, I want to see my requests and drop one I no longer need, so that I know where I stand and do not hold dates I will not use.

Acceptance criteria

- Given I have loans, when I open my loans, then I see each one with its dates and status, newest first.
- Given a request was rejected, when I open my loans, then I can see the reason.
- Given a Pending request, when I cancel it, then the status becomes Cancelled and those dates are free again straight away.
- Given an Approved loan, when I open it, then there is no cancel button.

Epic 4 — Approvals and Returns · 18 points · Needs Epic 3

The rest of Workflow B. The decision is where the overlap rule is checked for the last time.

Story 4.1 — See the pending request queue · 5 pts · Must · R12

As a technician, I want one list of everything waiting, so that nothing sits unanswered.

Acceptance criteria

- Given there are pending requests, when I open the queue, then I see student, headset and dates, oldest first.
- Given the queue is empty, when I open it, then I see a message, not a blank page.
- Given I am a student, when I call the queue route, then the server returns 403.

Story 4.2 — Approve or reject a request · 8 pts · Must · R12, R13

As a technician, I want to decide on a request, so that the student knows where they stand.

Acceptance criteria

- Given a pending request with no clash, when I approve it, then the status becomes Approved and the student can see it.
- Given a clash appeared since the request was made, when I approve it, then it is refused and the status stays Pending.
- Given I reject a request, when I save, then a reason is required and the student can see it.

Sub-tasks

- Approve route, reject route with reason, second overlap check inside approve, clash test.

Story 4.3 — Record collection and return · 5 pts · Should · R12

As a technician, I want to record the handover and the return, so that the lab knows who has what.

Acceptance criteria

- Given an approved loan, when I record collection, then the status becomes Collected with a timestamp.
- Given a collected loan, when I record the return, then the status becomes Returned and I can mark the condition as Good or Damaged.
- Given a returned loan, when a student searches those dates, then the headset is free again.

1.9 Sprint Plan

Two sprints, two weeks. Once a sprint starts, nothing is added to it. Both sprints hold Must work only. The one Should story is left for a third sprint.

Sprint length	3 weeks. At `15` hours a week, a two-week sprint cannot hold a whole workflow, so there would be nothing to show at the first review.
Team	`1` person. The jobs below are all done by the same person.
Capacity	45 hours a sprint.
Velocity	Sprint 1 was planned at about one hour a point. Sprint 2 uses the real Sprint 1 number.
Board	Jira. To Do, Blocked, Blockers, In Progress, In Review, Testing, Done.

Who owns what

One person does all the work, so ownership is shown as the job being done. The reviewer job is what catches the mistakes the developer job made.

Job	What it covers	Who
Developer	Writes the code and the tests. Makes the branch and pushes the commits.	Ali rezaeimanesh
Reviewer	Opens the pull request, writes the self-review comment, checks no secrets were committed, merges.	Ali rezaeimanesh

Job	What it covers	Who
Product owner	Sets the priority and accepts stories against their acceptance criteria.	Ali rezaeimanesh

Definition of Done

A story is only Done when all five are true. Points are not counted before that.

- Every acceptance criterion has been run by hand and passes.
- New server rules have unit tests and those tests pass.
- The branch is merged into main through a pull request with a self-review comment.
- No password, key or connection string is in the commit.
- The change is on EC2 and works on the public URL.

Sprint 1: Accounts and inventory

Dates	15 Aug – 26 Aug
Goal	A technician can log in and add headsets. A student can log in and see them.
Committed	41 points
Delivered	33 points

ID	Item	Pts	Outcome
1.4	Set up the database and EC2	5	Done
1.1	Sign up	5	Done
1.2	Log in and log out	5	Done
1.3	Role check middleware	5	Done
1.5	Write the README and deployment steps	5	Done
2.1	Add a headset	5	Done
3.1	Browse the headset list	3	Done
2.2	Edit a headset and set its status	5	Not started. Carried to Sprint 2
2.3	Retire a headset	3	Not started. Carried to Sprint 2
	Total committed	41	33 delivered

1.3 was built before the inventory work, even though users never see it. Every story in Epic 2 has a criterion that says a student gets a 403, and there is nothing to test that against until the middleware exists.

Blockers

Item	Waiting on	Why it was stuck	How it cleared
1.1 Sign up	1.4 Database	No account can be saved without a database.	Atlas connected on day 2.
2.1 Add a headset	1.3 Role middleware	The 403 criterion could not be tested.	1.3 done on day 8. Came out of Blocked the same day.
1.4 Database	Outside the team	The starter app would not run on Node 25. A dependency of jsonwebtoken uses SlowBuffer, which Node 22 removed.	Upgraded jsonwebtoken. Half a day lost.

Risks

ID	Risk	Chance	Impact	What we do about it
RK1	The starter app does not run on the current Node version.	Medium	High	Pin the version and upgrade the dependency that breaks. This one happened.
RK2	Atlas blocks a machine because its IP is not on the access list.	Medium	Medium	Add "MyIP" to security group with TCP custom and related port.
RK3	Role checks end up in the interface only and not on the server.	Low	High	1.3 has its own test that sends a student token to a technician route.
RK4	A secret gets committed early and stays in the git history.	Low	High	`.gitignore` was written before the first line of code.

Review

Committed	41 points
Delivered	33 points
Real velocity	33 points
Carried	2.2 Edit a headset and 2.3 Retire a headset, 8 points
Goal met	Yes. Both roles log in, headsets can be added, and students can see the list.

Retrospective

- Went well: building the role middleware before the inventory work, meant every later story could test its 403 case straight away.
- Went badly: half a day went on a Node version problem in the starter app that nobody had checked for.
- Changed for Sprint 2: put the two workflows first and let the smaller inventory stories slip if the sprint runs short.

Sprint 2: Availability, requests and approvals

Dates	26 Aug – 1 Sep
Goal	Workflow A runs end to end. A student can search, request and see the answer. A technician can see the queue and decide.

Committed	47 points, which is the 39 planned plus 8 carried from Sprint 1
Delivered	29 points

ID	Item	Pts	Outcome
3.2	Search availability for a date range	8	Done
3.3	Request a loan	8	Done
4.1	See the pending request queue	5	Done
4.2	Approve or reject a request	8	Done
3.4	See my loans and cancel a request	5	Part done. My loans works, cancel not built
1.6	Unit tests to 70% coverage	5	Not started
2.2	Edit a headset and set its status	5	Not started
2.3	Retire a headset	3	Not started
	Total committed	47	29 delivered

3.2 went first because the rest of the sprint leans on it. The overlap query it builds is used again by 3.3 and again by 4.2.

Blockers

Item	Waiting on	Why it was stuck	How it cleared
3.3 Request a loan	3.2 Search availability	The request runs the same overlap check the search runs. Writing it twice would let the two drift apart.	3.2 done on day 5. The query went into one function and both call it.
4.1 and 4.2	3.3 Request a loan	Nothing to queue and nothing to approve until requests exist.	3.3 done on day 8.
Deployment	Outside the team	The EC2 instance serves correctly on its private address, but its public IP does not route. Nothing outside AWS can reach it.	Not cleared. Reported to the unit coordinator. See the note below.

Risks

ID	Risk	Chance	Impact	What we do about it
RK5	Two approvals both pass the overlap check before either one saves, and the headset gets double booked.	Medium	High	The approve step checks the database again and refuses if it clashes. See D5.
RK6	Time zones move a loan by a day, so a search and a loan record disagree.	Medium	Medium	Dates are stored as UTC midnight and compared as whole days.

ID	Risk	Chance	Impact	What we do about it
RK7	The coverage target is missed because tests are left to the last week.	Medium	Medium	This one happened. 1.6 was never started and the target was not met.
RK8	The EC2 public URL is not reachable during marking.	Low	High	This one happened. The cause was found and reported, but it is outside the team to fix.

Review

Committed	47 points
Delivered	29 points
Real velocity	29 points
Not delivered	3.4 cancel, 1.6 unit tests, 2.2 edit a headset, 2.3 retire a headset. 18 points
Coverage reached	Not measured. Story 1.6 was not started.
Goal met	Partly. Workflow A runs end to end on the server and both roles work. The public URL is not reachable, which is a deployment problem rather than a code one.

Retrospective

- Went well: the overlap query went into one function and all three callers use it, so the rule cannot drift apart between search, request and approve.
- Went badly: unit tests were planned as their own story at the end of the sprint and never got started. Writing them alongside each story would have got some of them done.
- Went badly: two days went on the EC2 public IP problem before it turned out to be an account limit that could not be worked around.
- Changed going forward: put testing inside each story rather than in one card at the end.

What is left after Sprint 2

62 of the 85 points are delivered. 23 points are left, across five items.

ID	Item	Pts	Why it was dropped
3.4	Cancel a pending request	5	My loans is built and shows every status. Cancelling is the part that is missing.
1.6	Unit tests to 70% coverage	5	Left too late in the sprint. The overlap cases were checked by hand instead.
2.2	Edit a headset and set its status	5	A technician can add and list headsets, which is what the two workflows need.
2.3	Retire a headset	3	The query already hides retired headsets. Only the button is missing.
4.3	Record collection and return	5	A Should item. Both handovers happen at the counter in person.

ID	Item	Pts	Why it was dropped
	Total	23	

The brief asks for one complete workflow, and that is what was built. Workflow A runs from sign up through to a student seeing the decision, and the technician side does everything Workflow A needs from it. The five items above are all things the two workflows can run without.

Note on the deployment

The application runs on the EC2 instance. It answers on the private address and Node is bound to every interface. The auto-assigned public IP does not route, so nothing outside AWS can reach it, and the instance cannot even reach itself through that address.

The security group allows inbound HTTP from every address, the subnet routes to an active internet gateway, and the network ACL allows all traffic. An Elastic IP would fix it, but the account has reached its address limit. This was reported to the unit coordinator.

1.10 Decision and Change Log

Rows are added, never edited. If a decision is undone later, the reversal gets its own row.

ID	What changed	Why	Evidence looked at	Affected
D1	The project changed from a game playtesting platform to an XR headset borrowing system.	The borrowing system has a real scheduling rule in the middle of it. Two loans of the same headset cannot overlap, and a spreadsheet cannot do that.	The approved project list, item 61. Confirmed by the tutor by email.	All of Task 1, the backlog, every diagram
D2	Two roles only, Student and Technician. A Lab Manager role was dropped.	A manager role would only read data. It adds screens but no new rule to test.	The permission table in 1.3. Lab manager stays a stakeholder but is not a user role.	1.2, 1.3, R3, Epic 1
D3	Workflow A was picked for the behaviour view, drawn as an activity diagram rather than a sequence diagram.	Workflow A has the availability check in it, so it shows more of how the system works. Workflow B is mostly a status change. Activity was chosen over sequence because the only behavioural example in the unit material is an activity diagram, so the notation can be matched to something taught.	R13 and the seven overlap tests in Story 3.2. The activity diagram on Week 1 slide 70.	Task 2, the traceability matrix
D4	Delete became a status change instead of removing the record.	Loan history has to survive. Deleting a headset would	The criterion on Story 2.3 that says old loans still show the retired headset.	Stories 2.3 and 3.4, the block

ID	What changed	Why	Evidence looked at	Affected
		leave old loans pointing at nothing.		definition diagram
D5	A second overlap check was added inside the approve step.	A clash can appear in the gap between a student searching and a technician approving. One check is not enough to keep R13 true.	Walking through Workflow A step by step showed the gap.	Story 4.2, R13, SC1, the activity diagram
D6	Sprints are 3 weeks, not 2.	At `15` hours a week a two-week sprint cannot hold a whole workflow.	80 Must points against 45 hours a sprint. The tutor confirmed at least two iterations are expected.	1.9
D7	Sign up offers both roles, and picking Technician asks for a staff code checked against an environment variable.	The scope said the role is picked at sign up, but sign up only ever made students, so no technician account could exist. A plain dropdown was rejected because anyone could make themselves a technician.	R14 and the wrong-role tests in Story 1.3. A marker also has to be able to make a technician account during marking.	1.3, 1.5, R2, Story 1.1, the requirement diagram
D8	Epics went from 5 to 4 and the backlog was merged.	Deployment, docs and testing had no user story attached, so they moved into Foundations with accounts and roles. Stories that share a screen or a form were merged.	The tutor confirmed there is no recommended number of epics.	1.8, 1.9, 2.2
D9	The accounts tree was split into eight requirements. Roles became a requirement of its own. The non-functional group grew from two to four.	Signing up, authenticating, logging in and logging out are built and tested separately, so one requirement covering all of them could not be verified as one thing. Student and Technician were used all through the document but never written down, so nothing said there are exactly two or that an account has one. The old security requirement held three rules in one box.	Tutor feedback asking whether the relationships are correct and sensible. The role table in 1.3 and criteria SC1, SC2, SC5 and SC7 already set the four non-functional targets.	1.6, 1.7, 1.8, 1.9, 2.1, 2.2, the requirement diagram

TASK 2 — SYSTEM DESIGN

2.1 The Diagrams

Three diagrams. They are all in one draw.io file, one per tab, and the link is on the cover page. The requirement diagram shows what the system must do. The block definition diagram shows what the system is made of. The activity diagram shows how a student borrows a headset from start to finish. No two of them show the same thing.

2.1.1 Requirement Diagram

Sixteen requirements in two groups under one root, joined by containment. The Functional boxes are the three starting points: R1 Sign up, R3 Roles and R9 Manage headsets. Everything else hangs off those.

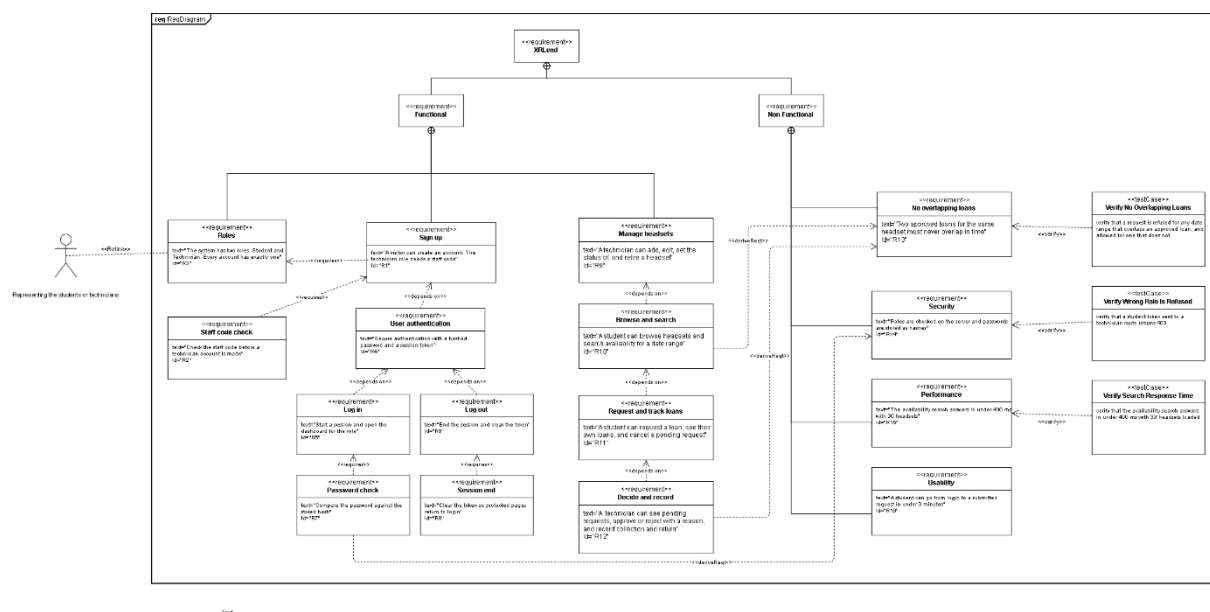
Sign up requires Roles, because an account cannot be made without a role to give it. The staff code check is required before a technician account exists. R4 depends on R1, R5 and R6 both depend on R4, and R7 and R8 sit underneath as the parts log in and log out need.

On the other branch, R10 depends on R9 because you cannot search headsets that do not exist. R11 depends on R10 and R12 depends on R11. That chain is also the order the sprints build them in.

Four derive links cross to the non-functional side. R10 and R12 come from R13, because searching and approving are the only two places the overlap rule is enforced. R7 comes from R14. R14 comes from R3.

The two actors sit outside the frame and both point at R3 with a refine link. R3 says two roles exist. The actors are what those roles are, so R3 is the only sensible thing for them to point at.

Three test cases say how the non-functional requirements are proved. R16 has none, because it is measured by timing a person through the prototype rather than by a unit test.

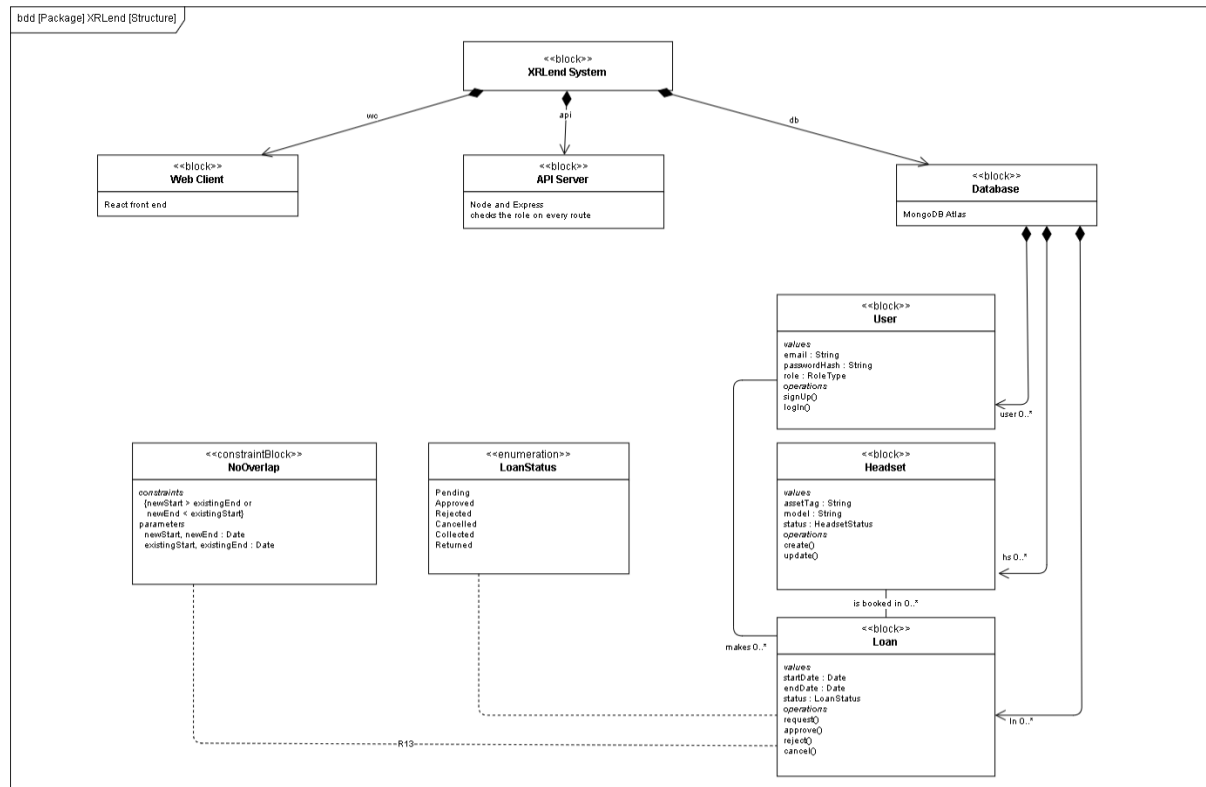


2.1.2 Block Definition Diagram

This is the system design diagram. The XRlend System block is made of a Web Client, an API Server and a Database. The API Server opens up into RoleMiddleware and three controllers. The Database opens up into User, Headset and Loan, each with its values and operations.

Four enumerations fix the values that matter. RoleType holds the two roles from R3. LoanStatus is the important one, with six values a loan can be in.

A constraint block called NoOverlap is attached to Loan. It holds R13 as an expression: a new loan is only allowed if it finishes before the existing one starts, or starts after it finishes. That block is what makes this a SysML diagram rather than a plain class diagram.

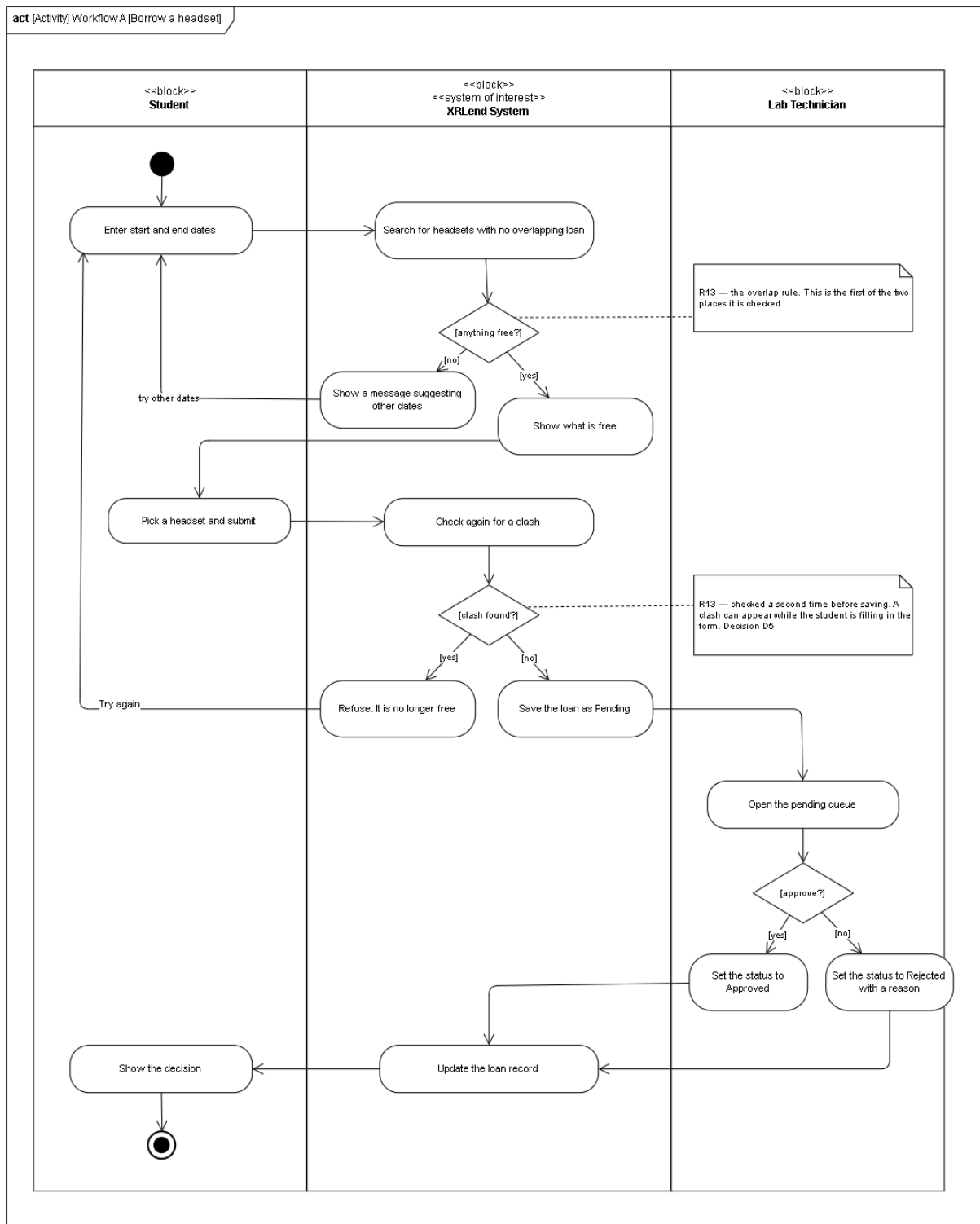


2.1.3 Activity Diagram — Workflow A

Workflow A was picked over Workflow B in decision D3. The diagram has three swimlanes, one per block: Student, XRLend System and Lab Technician. XRLend System is marked as the system of interest, the same way the activity diagram in the Week 1 lecture marks the ESS block.

It starts when the student enters their dates and ends when they see the decision. Three decision nodes carry the branches. Is anything free, which loops back to the dates if not. Was a clash found, which refuses the request and stops. And approve or reject, which sets the status either way.

The first two decision nodes are the same rule checked at two different moments. A clash can appear while the student is filling in the form, so checking once at search time is not enough. That is decision D5, and the two diamonds are what make it visible.



2.2 Traceability Matrix

Every requirement is followed from the backlog, the design, the interface, and the commit. Five rows are marked as not built. Section 1.9 says why each one was dropped.

Req	Story	Design element	Figma frame	Commit	Evidence
R1	1.1	User block, signUp()	create acc	e5c5267	A new account is created and lands on the student dashboard
R2	1.1	AuthController, staff code check	create acc	e5c5267	A wrong staff code returns 403 and makes no account
R3	1.1	RoleType enumeration, User.role	create acc role picker	e5c5267	Every account row carries exactly one role
R4	1.1	User.passwordHash, token issue	create acc, login	e5c5267	The token is returned and accepted by protected routes
R5	1.2	User.login(), AuthController	login, login error	a264ca6	A student reaches Find, a technician reaches Lab overview
R6	1.2	AuthController, token clear	Account tab	a264ca6	Protected pages return to login after logging out
R7	1.1	Compare password action	Not a screen. Server side only	e5c5267	The database shows a hash, never the plain password
R8	1.2	RoleMiddleware, token check	Not a screen. Server side only	a264ca6	An old token is refused after logout
R9	2.1	Headset block, create()	T headsets	f7bcff6	A headset is added and shows in the list. Edit and retire not built
R10	3.1, 3.2	HeadsetController, Headset.isAvailable()	Find a headset, out of date	381e9ba	The search returns only headsets that are free
R11	3.3	Loan block, request()	send request, Request sent, My loans	c6127af	A request is saved as Pending. Cancel not built
R12	4.1, 4.2	LoanController, approve(), reject()	T requests, T reject	9852b6e	The status changes and the student can see it
R13	3.2, 3.3, 4.2	NoOverlap constraint block, both decision nodes	Find a headset	381e9ba, c6127af, 9852b6e	A clashing request is refused with a 409 at request and at approve
R14	1.1, 1.3	RoleMiddleware block, the XRLend System lane	Not a screen. Server side only	f62e350	A student token sent to a technician route returns 403
R15	3.2	Headset.isAvailable()	Find a headset	381e9ba	The search answers immediately with the lab's 30 headsets
R16	3.2, 3.3	The Student lane, start node to Show the decision	login through to Request sent	a264ca6	A student goes from login to a submitted request in under 3 minutes

R9, R11 and R15 are partly delivered. Editing and retiring a headset, cancelling a request, and measuring the search time were all dropped in Sprint 2. Section 1.9 lists them with the reason for each.

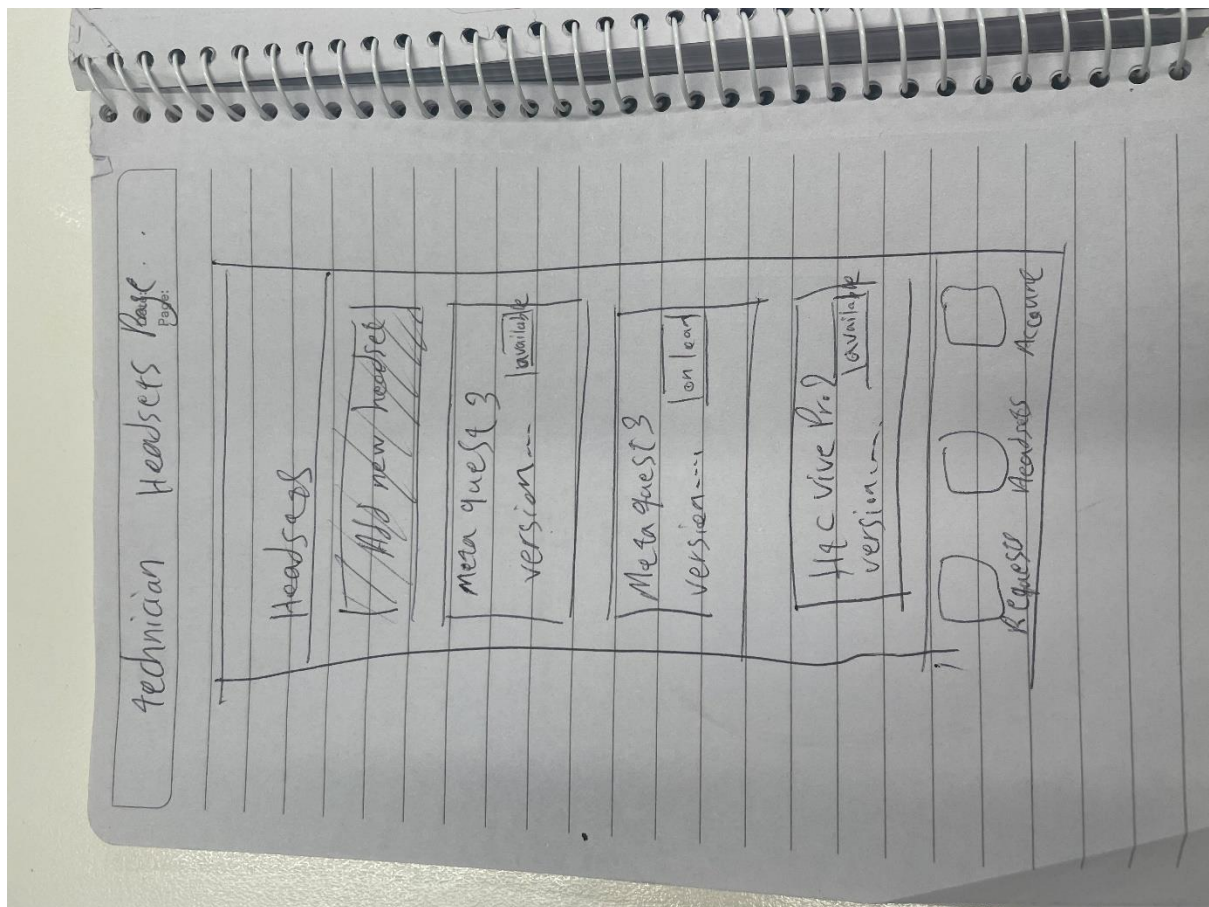
The evidence column describes what was checked on the running application. The EC2 public address does not route, which is written up at the end of 1.9, so the checks were run against the server itself and on a local build.

TASK 3 — UI/UX DESIGN

3.1 Low-fidelity wireframes

The screens were drawn in grey first. No colour, no fonts. That pass was only about where things sit on the page and how you get from one screen to the next. Every frame is 390 by 812, which is a phone. A student booking a headset is more likely to be on a phone than at a desk.

The wireframes settled two things early. Both roles share the same sign up and log in screens, and the role picked at sign up decides which UI you will have. And every list is empty at first, because a new lab has no headsets and a new student has no loans.



3.2 High-fidelity prototype

The high-fidelity screens use Inter for all the text. Primary buttons are near-black, #04000b. Cards are white with a #cdcdcd border on a light grey background.

Almost all the colour goes on status. Six chips cover the six loan statuses. Four of those are reused for headset status, so the inventory screens needed no new colours.

Six things are built as components rather than copied around: the top bar with its role chip, the bottom tab bar, the text field, the button, the list row and the status chip. The list row shows up on four screens and the chip on five.

3.3 The four states

The brief asks for a normal, empty, error and success state. Each one is its own frame, and the frame name says which state it is. That way you do not have to open every screen to find them.

State	Frame	What it shows
Normal	Find a headset	Three headsets free for the chosen dates, each with a Request button
Empty	Find a headset — empty	Nothing free for those dates, with a suggestion to try a shorter stay
Validation	Find a headset — validation	The return date is before the pick-up date. Nothing is sent to the server
Error	login error	The email and password do not match. It never says which of the two was wrong
Success	Request sent	The request is saved as Pending, with the headset and dates confirmed

The prototype is clickable and covers both roles. A student goes from log in, through the search and the request, to the confirmation and then their loan list. A technician goes from log in to the lab overview, the inventory and the request queue. The reject sheet slides up over the queue. The view-only link is on the cover page.

TASK 4 — GIT VERSION CONTROL

4.1 Branches and pull requests

Every piece of work went on its own branch, named after the Jira issue. The availability search was built on XR-17-search-availability, and its commit message starts with XR-17. So the board and the repository can be read side by side without guessing which is which.

Nine pull requests were opened and merged into main. Each one got a comment on the diff, not just an approval.

The review on the role middleware branch is the clearest one. It spotted two missing return statements before `res.status(401)` in `authMiddleware.js`. Without them, a failed token would try to send headers twice and throw an error. The fix went in as a second commit on the same branch, so the comment and the fix are both in the history.

4.2 Commit history

Commit	Jira	What it did
e5c5267	XR-6	Role field and staff code check on sign up
f62e350	XR-8	Role check middleware, and the auth middleware fix from the self-review

Commit	Jira	What it did
f7bcff6	XR-13	Headset model, controller and routes, technician-only create
381e9ba	XR-17	Loan model and the availability search that enforces the no-overlap rule
c6127af	XR-18	Loan request with the second overlap check, my loans, pending queue
9852b6e	XR-21	Approve and reject, with a third overlap check before saving
a264ca6	XR-16	The XRLend pages, the auth token interceptor and role routing

The release is tagged v1.0. Nothing was rebased, squashed or backdated.

The first few commits are messier than the rest. The repository started out as somewhere to keep the report and the diagrams, before there was any code. Those commits were left alone.

4.3 README

The README covers setup, an architecture summary, the known limitations and the deployment. It lists the same links as the cover page, so the repository works on its own.

The limitations say what is actually true. Six of the sixteen backlog stories are built in the application. There is no HTTPS. And npm audit reports problems inside react-scripts, which were left alone because upgrading breaks the build.

TASK 5 — SAMPLE APPLICATION AND EC2 DEPLOYMENT

5.1 What was built

Workflow A runs end to end. A student signs up, logs in, picks two dates, sees only the headsets that are free, requests one, and later sees the decision on their loan list. A technician signs up with the staff code, adds headsets, opens the pending queue, and approves or rejects with a reason that will be shown to the student.

The overlap rule is written once and called from three places. `utils/availability.js` holds the query. The search calls it, the request calls it, and the approve step calls it. The last two return a 409 and leave the loan as it was.

That is decision D5 turned into code. A clash can appear while a student is filling in the form, and again between the request and the decision. Checking once at search time is not enough.

5.2 Deployment

Instance	i-0e07c3de4b0f61dad (Ali_rzm), Ubuntu, ap-southeast-2
Stack	Node 20, Express, MongoDB Atlas, React build served as static files
Port	5001
Process	pm2, with pm2 startup so the app comes back after a reboot
Deploy	Manual. git pull, npm run build in the frontend, pm2 restart. CI/CD is out of scope

One Express process serves both the API and the React build, so only one port has to be open. The frontend and the backend are not deployed as two separate things.

5.3 Security

No password, key or connection string is in the repository. The .gitignore was written before the first line of code, so .env has never been committed. backend/.env.example lists the four keys with nothing filled in, and the README says how to fill them.

The security group allows SSH on port 22 and the application port, from named addresses only. Rules with a source of 0.0.0.0/0 get removed automatically by the account policy, so every machine that needs access has to be added by its address. That is tighter than a public web app would normally be. It also means the deployed app can only be reached from a machine that has been added.

The app runs as the ubuntu user, not as root. Passwords are stored as bcrypt hashes. Every protected route checks the role on the server, not just in the interface.

TASK 6 — GenAI disclosure and report

6.1 GenAI Use Declaration and Evidence Log

I used GenAI tools during this project to support different parts of my work. I used them for writing, design ideas, coding, organising information, and solving technical problems. I reviewed the information and code provided by the tools and made changes where necessary to suit my project.

GenAI	How I used it	How I used the output
Grammarly	I used Grammarly for the report and other written parts of the project to check	I reviewed the suggestions and used them to correct grammar and improve the writing.

GenAI	How I used it	How I used the output
	grammar, spelling, punctuation, and sentence structure.	
Claude – Colour palette	I asked Claude to generate colour palette ideas for my design.	I used the suggestions as ideas and selected or adjusted the colours based on my design.
Claude – Frontend code	I asked Claude to generate frontend code in JavaScript based on my design in Figma.	I reviewed the generated code, changed it where necessary, and tested it in my project.
Claude – Backend code	I asked Claude to generate and help with backend code for my project, including code related to the database and communication between the frontend and backend.	I reviewed the generated code, made changes to match my project requirements, and tested it before using it.
Claude – GitHub commit messages	I asked Claude to help me write suitable messages for my GitHub commits.	I used the suggested messages when they correctly described the changes I had made.
Claude – Tables	I asked Claude to put some project information into tables.	I checked the information and used the tables to present it in a clearer way.
Claude – Technical problems	I asked Claude for help when I faced technical problems during development, such as an issue with the MongoDB SRV connection link.	I used the suggestions to understand the possible cause of the problem and find possible solutions. I then tested the solutions and made the required changes.

6.2 Individual Reflection

6.2.1 Challenge

One challenge I faced during the project was connecting my application to MongoDB. I had an issue with the MongoDB SRV connection link, which meant that my application could not connect to the database correctly after I changed my network. This caused problems when I was testing features that needed to save or retrieve information from the database.

6.2.2 Response

I first checked my database settings and the error messages to understand what was causing the problem. I also asked Claude for help to understand the possible causes of the MongoDB SRV connection problem. I used the information to check my connection string and configuration. I then tested the possible solutions and made the required changes to my project.

6.2.3 Evidence

After making the changes, the application was able to connect to MongoDB successfully. I was then able to test the features that required communication with the database. The final working application and the project code show that the problem was resolved.

6.2.4 Future Improvement

In a future project, I would test the database connection earlier before developing other features that depend on it. I would also spend more time understanding the database configuration and connection process at the beginning of the project. This would help me find connection problems earlier and save time during development. I would also make more use of the official documentation when dealing with technical configuration issues.